

Digit DP

Problemas que resolvem

- **Identificação:** Intervalos gigantes $[L, R]$ com $R \leq 10^{18}$ ou dados como string (10^5 dígitos).
- **Objetivo:** Contar propriedades, somar dígitos, ver paridade, diferença absoluta, tudo que dependa da **estrutura dos dígitos** e não do valor absoluto do número.
- **Complexidade:** $O(\log_{10}(N) \times \text{Estados})$.

Código Padrão

```
int dp[20][11][2][2]; // [index][state][tight][ldz]

int pd(int index, int state, int tight, int ldz) {
    if(index == n) return /* condicao base */;

    if (dp[index][state][tight][ldz] != -1) return dp[index][state][tight][ldz];

    int ub = tight ? r[index] - '0' : 9;
    int ans = 0;

    for (int digit = 0; digit ≤ ub; digit++) {
        // if(!ldz && condicao_invalida) continue;
        int new_tight = tight && (digit == ub);
        int new_ldz = ldz && (digit == 0);
        int new_state = state; // atualiza seu estado

        ans += pd(index + 1, new_state, new_tight, new_ldz);
    }
    return dp[index][state][tight][ldz] = ans;
}
```

Código usando **above** e **under**

Quando tem limite inferior e superior, e não da pra fazer **count(r) - count(l-1)**
As strings tem que ter o mesmo tamanho, então da pra adicionar **0** na `string l`

```
int lb = under ? l[index] - '0' : 0;
int ub = above ? r[index] - '0' : 9;
for (int digit = lb; digit ≤ ub; digit++) {
    int n_above = above && (digit == ub);
    int n_under = under && (digit == lb);
}
```

Tricks

Cortar a Flag da dimensão (`tight` ou `above/under`)

A flag para saber se estamos nos limites dos números pode ser cortada para apenas um `if` dentro da recursão.

O caminho onde `tight == 1` é visitado no máximo 1 vez por índice. Guardar isso no `memo` é inútil.

Corta o uso de memória pela metade

```
// Retorno rápido se não estiver colado no teto
if (!tight && dp[index][state] ≠ -1) return dp[index][state];
// ...
// Na hora de salvar, só salvo os universais
if (!tight) dp[index][state] = ans;
```

Otimização do `memset`

Como a gente cortou o `tight`, o meu estado `dp[index][state]` passou a significar "*De quantas formas posso preencher um sufixo de tamanho $N - index$* " independente do limite R da query.

Com isso, da para fazer `memset(dp, -1, sizeof dp)` só uma vez na `main()`.

Precisa fazer padding para as strings terem todas o mesmo tamanho

Complexidade total amortece pra $O(\text{States} + \text{Testes} \cdot \log R)$.

```
int count(int x){
    s = to_string(x);
    int pad = 19 - s.size(); // tamanho do vetor dp
    if (pad > 0) s = string(pad, '0') + s;
    n = s.size();
    return pd(0,0,1);
}

void solve(){
    int l, r;
    cin >> l >> r;
    memset(dp, -1, sizeof(dp));
    cout << count(r) - count(l-1) << endl;
}
```

Lidando com leading zeros (`ldz`)

O número `00042` tem 5 dígitos na string, mas pro problema matemático só tem 2. Zeros à esquerda não podem contar como para a solução.

Da para fazer uma flag `ldz` que vai marcando se é leading zero.

```
int new_ldz = ldz && (d == 0);
// Só computo a lógica principal se for número real:
if (!new_ldz) {
    // ... conta adjacência, adiciona diferença
}
int next = new_ldz ? 10 : digit;
```

Em problemas onde a gente tá considerando o último dígito, dá para mandar o dígito 10 ou -1 como padrão quando ainda estamos em ldz, para não atrapalhar a contagem de zeros.

Usando mask para guardar números únicos

Problema pede "no máximo K dígitos distintos" ou algo de diversidade.
Máscara de 10 bits (de 0 a 1023). O bit i ligado significa que usei o dígito i .

```
int new_mask = mask;
if (!new_ldz) new_mask |= (1 << digit);
```

Inverter o número (De trás pra frente)

Em problemas com muitos casos de teste o `memset` constante na DP vai dar **TLE**. Se o dígito mais significativo (MSB) estiver sempre no `index = v.size() - 1`, o estado `dp[3]` pode representar 10^3 em um número de 4 dígitos, mas 10^5 em um de 6. Inverter o vetor resolve isso. Ao preencher o vetor com `x % 10`, a posição `0` será sempre a unidade (10^0), a posição `1` a dezena (10^1), e assim por diante.

Como a potência de 10 associada a cada `index` é fixa, os resultados de subproblemas (sufixos) são os mesmos para qualquer N . Podemos rodar o `memset` uma única vez no `main`.

Cuidado ao usar essa técnica em problemas onde o estado da DP depende do **tamanho total do número** (ex: "números que possuem o dígito 7 na posição central"). Se o tamanho importa, o seu estado da DP precisaria de uma dimensão extra para `total_len`, ou você teria que voltar ao `memset` por query.

```
vector<int> v; //lembrar que aqui vira vetor e não string
int pd(int index, int tight, int mask, int ldz){
    if (index == -1) return ((__bit_width(mask) - 1) == __popcount(mask));
    if (!tight && dp[index][mask][ldz] != -1) return dp[index][mask][ldz];

    int ub = tight ? v[index] : 9;
    int ans = 0;
    int sum = 0;

    for (int digit = 0; digit ≤ ub; digit++){
        int new_tight = tight && (digit == ub);
        int new_ldz = ldz && (digit == 0);
        int new_mask = mask;
```

```

        if (!new_ldz) new_mask |= (1 << digit);
        ans += pd(index - 1, new_tight, new_mask, new_ldz);
    }
    if (!tight) dp[index][mask][ldz] = ans;
    return ans;
}

```

```

int count(ll x){
    v.clear();
    while(x) {
        v.push_back(x % 10);
        x /= 10;
    }
    /* [L,R]
    int count(int x, int y){
    v2.clear();
    while(x) {
        v2.push_back(x % 10);
        x /= 10;
    }
    v.clear();

    while(y) {
        v.push_back(y % 10);
        y /= 10;
    }
    while(v2.size() < v.size()) v2.push_back(0LL);
    */
    n = v.size();
    return pd(n-1,1,0,1);
}

```

```

void solve(){
    ll r;
    cin >> r;
    cout << count(r) << endl;
}

```

```

signed main(){
    winton;
    memset(dp, -1, sizeof(dp));
    int t;
    cin >> t;
}

```

```
while(t-->0) solve();
```

```
}
```